
DOCUMENTO DE REQUISITOS — APP FUTEBOL (HARMONIA FC)

1. OBJETIVO DO SISTEMA

Aplicação web para gestão de jogos recorrentes de futebol entre amigos, com foco em:

- organização de presença
 - controle de mensalidade
 - gestão de campeonato interno
 - gestão do grupo da carne (churrasco de toda quarta-feira)
 - automação de sorteio de times
 - redução de erro humano na organização
-

2. PERFIS DE USUÁRIO

2.1 Jogador

Pode:

- fazer login (por telefone)
- confirmar presença
- visualizar jogo
- ver status de mensalidade
- acompanhar campeonato
- acompanhar tabela da carne
- visualizar notificações

Não pode:

- alterar configurações
 - alterar jogo
 - encerrar campeonato
-

2.2 Administrador

Tudo que jogador faz +

- configurar jogo

- definir data/hora
- definir máximo de jogadores
- controlar vencimento da mensalidade
- abrir/fechar inscrições
- executar sorteio
- limpar sorteio
- encerrar campeonato
- enviar notificações
- adicionar e excluir jogadores
- configurar tabela da carne, podendo adicionar e excluir jogadores da tabela

Regra:

admin = player.is_admin OU telefone em ADMIN_PHONES

2.3 Jogador Carne

Pode:

- fazer login (por telefone)
- acompanhar campeonato
- acompanhar tabela da carne
- visualizar notificações

Não pode:

- alterar configurações
- alterar jogo
- alterar tabela da carne
- encerrar campeonato

3. 🧠 ENTIDADES DO SISTEMA

3.1 Player

```
{
  "id": string,
  "name": string,
  "phone": string,
  "position": "zag" | "meia" | "atk",
  "mens_ok": boolean,
  "is_admin": boolean
}
```

3.2 Game

```
{
  "game_date": "YYYY-MM-DD",
  "game_time": "HH:mm",
  "max_players": number,
  "mens_expire_date": "YYYY-MM-DD",
  "open": boolean,
  "sort_result": object | null
}
```

3.3 Confirmation

```
{
  "player_id": string,
  "confirmed": boolean,
  "timestamp": date
}
```

3.4 Championship

```
{
  "id": string,
  "start_date": date,
  "end_date": date | null,
  "closed": boolean,
  "ranking": [...]
}
```

3.5 Notification

```
{
  "id": string,
  "message": string,
  "created_at": date
}
```

4. TELAS DO SISTEMA

4.1 Login

- identificação por telefone e senha
 - opção de cadastro (caso não tenha cadastro ainda)
 - persistência via sessionStorage
 - acesso via identidade do telefone (faceID, etc.)
-

4.2 Cadastro

- Nome
 - Telefone
 - Data de Nascimento
 - Jogador ou apenas carne
 - Posição de campo (se jogador)
 - Senha
 - Repetir senha
-

4.3 Home

Exibe:

- próximo jogo
- data/hora
- vagas preenchidas
- status da mensalidade
- botão confirmar presença

Regras:

- não permite confirmar se:
 - mensalidade vencida
 - jogador não está em dia
-

4.4 Jogadores

- lista de jogadores
 - mensalidade (em dia / em atraso)
 - status (confirmado / não confirmado)
 - posição (zag / meia / atk)
-

4.5 Campeonato

- ranking atual
 - histórico
 - status (ativo / encerrado)
-

4.6 Config (Admin)

Permite:

- alterar data e hora do jogo
 - definir máximo de jogadores
 - definir vencimento da mensalidade
 - abrir/fechar inscrições
 - executar sorteio
 - limpar sorteio
 - encerrar campeonato
 - informar e alterar pontuação por jogo e por jogador
 - enviar notificações
-

5. 💰 REGRAS DE MENSALIDADE

Funções existentes

- `isMensExpired()`
 - `canConfirm(player)`
 - `getMensalidadeHomeMeta()`
-

Regras

1. mensalidade vence dia 10
2. se vencida:
 - jogador não pode confirmar
3. status exibidos:

Situação	Status
em dia	Em dia
antes do vencimento	Atenção

vencido

Pendent
e

6. REGRAS DO JOGO

- limite de jogadores (**max_players**)
 - confirmação depende de:
 - jogo aberto
 - mensalidade OK
-

7. SORTEIO

Função:

doSort()

Regras:

- sorteia até **max_players**
 - usa apenas confirmados
 - exclui inadimplentes se mensalidade vencida
 - respeita posições:
 - zagueiro
 - meia
 - atacante
-

8. CAMPEONATO

- pode estar ativo ou encerrado
 - ao encerrar:
 - registra ranking
 - inicia novo ciclo
-

9. NOTIFICAÇÕES

- admin envia mensagem
- exibida para jogadores

10. 🧱 ARQUITETURA ATUAL (PROBLEMA)

- SPA em `index.html`
- JS parcialmente modularizado:
 - `auth.js`
 - `config.js`
 - `finance.js`

Problemas:

- alto acoplamento global
- dependência de variáveis globais
- risco alto de regressão
- fluxo de boot frágil
- responsabilidades misturadas

11. 🎯 REQUISITOS NÃO FUNCIONAIS

- não quebrar funcionalidades existentes
- manter UX simples
- mobile-first
- baixo tempo de carregamento
- sem dependência de build tools
- funcionamento offline parcial (mock)

12. 🚧 PROBLEMAS ATUAIS

- regressões frequentes
- dependência implícita entre arquivos
- dificuldade de evoluir
- ausência de separação clara de camadas

13. 🎯 OBJETIVO DA REESCRITA

- arquitetura modular real
- isolamento de responsabilidades
- boot previsível

- redução de regressões
 - facilidade de evolução
-

14. PONTOS EM ABERTO (VOCÊ PRECISA VALIDAR)

Responda isso para fechar o escopo:

1. Login continuará sem senha?

Não. É pré-requisito ter senha, como já explicado em sessões anteriores desse documento

2. Supabase permanece como backend?

Sim.

3. Campeonato precisa histórico completo?

Sim. Faremos upload e/ou aproveitaremos histórico do que está no supabase

4. Mensalidade será manual ou automatizada?

Automatizada, mas permite ao admin definir data mensal de vencimento

5. Notificação precisa push (ou só in-app)?

Só in-app, mas avisar sobre complexidade de implementar push

6. Precisamos de multi-jogo ou sempre 1 jogo ativo?

Sempre um jogo apenas.

7. Precisa controle financeiro completo ou só flag pago?

Apenas flag pago

RESUMO HONESTO

Você já tem:

- 👉 produto funcional
- 👉 regras claras
- 👉 usuários reais

O que falta:

👉 arquitetura confiável

PRÓXIMO PASSO

Você:

- revisa
- corrige
- adiciona

Depois eu:

- 👉 desenho a arquitetura nova
 - 👉 defino estrutura de código
 - 👉 proponho plano de migração sem quebrar o app atual
-

Se quiser, no próximo passo eu já posso:

👉 transformar isso em **especificação técnica pronta pra build** (nível senior)